cg-challenge-01-context

Code Context

Query: add a new LLM provider

Entry Points

- **Provider** (interface) - helix_code/internal/provider/provider.go:34
- **Provider** (interface) - docs/helix_qa/HelixQA_Integration/research/testdata/raw/pkg_llm_provider.go:40
- **Provider** (interface) - docs/helix_qa/HelixQA_Integration/research/testdata/raw/pkg_visionnav_provider.go:72

Code

Provider (helix_code/internal/provider/provider.go:34)

```
type Provider interface {
    GetType() ProviderType
    GetName() string
    GetModels() []llm.ModelInfo
    GetCapabilities() []llm.ModelCapability
    Generate(ctx context.Context, request *llm.LLMRequest)
        (*llm.LLMResponse, error)
    GenerateStream(ctx context.Context, request *llm.LLMRequest,
        ch chan<- llm.LLMResponse) error
    IsAvailable(ctx context.Context) bool
    GetHealth(ctx context.Context) (*llm.ProviderHealth, error)
    Close() error

    // Cognee integration methods
    SupportsCognee() bool
    InitializeCognee(config interface{}, options interface{})
        error
    GetModelName() string
    GetModelInfo() *llm.ModelInfo
    GetHardwareProfile() *hardware.HardwareProfile
}
```

Provider (docs/helix_qa/HelixQA_Integration/research/testdata/raw/pkg_llm_provider.go:40)

```
type Provider interface {
    // Chat sends a multi-turn conversation and returns the
    // assistant reply.
```

```
    Chat(ctx context.Context, messages []Message) (*Response,
        error)

    // Vision sends a screenshot (raw bytes) with a text prompt
    // and returns the assistant reply. Not all providers support
    // this — check SupportsVision before calling.
    Vision(ctx context.Context, image []byte, prompt string)
        (*Response, error)

    // Name returns the canonical provider identifier, matching
    // one of the Provider* constants.
    Name() string

    // SupportsVision reports whether this provider can process
    // image inputs via the Vision method.
    SupportsVision() bool
}
```

Provider (docs/helix_qa/HelixQA_Integration/research/testdata/raw/pkg_visionnav_provider.go:72)

```
type Provider interface {
    // Name identifies the provider (e.g. "anthropic-claude-
        opus-4",
    // "scripted-bank-yaml", "nop").
    Name() string
    // Decide returns the next action given the current
        observation
    // (most recent screen capture path + audio path). Returns
        error
    // if the provider can't decide (network failure, rate limit,
        etc.).
    // Validate() is called on the returned Decision before it's
    // returned — invalid Decisions surface as errors.
    Decide(ctx context.Context, obs Observation) (*Decision,
        error)
}
```